UNIVERSITY OF NORTH CAROLINA
CHARLOTTE

HireMeAI

Jeffery Sengsy
Siddhant Agarwal

Abstract:

Job seekers today face the challenge of crafting highly tailored application materials for each position, a process that is time consuming and difficult to scale. This project addresses that problem by developing an LLM-powered pipeline that accepts a user's background work history, skills, and preferences, retrieves relevant job listings, and analyzes each job description to extract key requirements and role-specific language. The system then dynamically rewrites the user's resume and generates a personalized cover letter that strategically aligns the candidate's experience with what each employer is seeking, producing materials that are contextually relevant and ATS-optimized without requiring manual effort from the applicant.

Background and introduction:

Jeffery Sengsy:

I'm a data professional currently in the final semester of my Master's in Data Science, bringing together more than eight years of experience in investment analytics and operations with a growing technical toolkit in SQL, Python, and machine learning. My career to date has been shaped by working at the intersection of finance and analytics translating complex datasets into insights that support investment decisions, streamline operational workflows, and inform strategy.
Through my graduate studies, I've expanded beyond traditional analytics into modern data science methods, including statistical modeling, predictive machine learning, and scalable data manipulation in Python. This combination of deep domain experience in investments and hands-on technical skills allows me to approach problems with both business context and

analytical rigor, understanding not just how to build a model or query a dataset, but why it matters to stakeholders and the bottom line.
I'm passionate about using data to bridge the gap between operational realities and strategic opportunities, and I'm excited to bring this blended skill set to roles where analytical depth, financial fluency, and technical capability intersect.

Siddhant Agarwal:
I am a student in the DSBA Masters program, aiming to complete by CY2026. I work as a Product Manager on M365 Copilot, helping improve RAG quality. Throughout my studies, I have been interested in finding places that Data Science can help improve core issues and pain points.

Finding jobs is a crucial topic and discussed in our classes, especially in the push of AI. Our motivation was to create an app that allows users to find jobs for different providers, and then get personalized/tailored resumes and CVs. Helping them represent their skills and capabilities in a format best for the job.

Methodology:

System Overview

HireMe.AI is structured as a multi-stage LLM pipeline with three core modules: (1) candidate profile ingestion, (2) job retrieval and ranking, and (3) tailored document generation. The system is deployed as a multi-page Streamlit application, with session state used to pass data across pages without re-computation.

3.1 Candidate Profile Ingestion

The system supports two profile input paths.

Upload path: Users upload a resume file in one of eight formats (.txt, .md, .docx, .pdf, .png, .jpg, .jpeg, .webp). Format-specific extractors handle each type: python-docx for Word documents (including table cell traversal), pypdf for text-layer PDFs, and a base64-encoded multimodal LLM call for image files. Text-based formats are read directly. All paths converge on a single parsing prompt sent to gpt-4.1-nano (temperature = 0.0), which extracts structured data conforming to the CandidateProfile Pydantic schema. The response is JSON-validated against the schema; a normalization pass coerces inconsistent field types (e.g., skills returned as

semicolon-delimited strings vs. arrays) before validation. A regex-based fallback independently extracts LinkedIn/GitHub URLs from raw text in case the LLM omits them.

Manual input path: Users fill a structured form with indexed widget keys (wexp_{i}_title, edu_{i}_degree, etc.) supporting dynamic addition and removal of work, education, and project entries. A two-layer persistence strategy keeps form state across page switches: entry dicts serve as stable storage, synced from widget keys on each render and restored to widget keys on return.

Both paths produce the same CandidateProfile dict, making downstream components agnostic to the input method.

Prompt injection defense: The parser prompt explicitly instructs the model to treat resume content as untrusted data and to ignore any embedded instructions that attempt to modify its role or output schema.

3.2 Job Retrieval

The job search module uses a LangChain agent (gpt-5-nano) with three tool integrations:

Adzuna API: structured search with salary filtering and employment type
The Muse API: category and experience-level filtering, company-specific browsing
JobSpy parallel scraping across LinkedIn, Indeed, Glassdoor, ZipRecruiter, and Google Jobs
The agent selects which tools to invoke based on the user's natural-language query, using query semantics to route to the most appropriate source or a combination of sources for broad coverage. Raw results from all three sources are tagged with a _source field and returned as a flat list for downstream ranking.

3.3 Retrieval-Augmented Ranking (RAG)

Because the job agent may return dozens of listings of varying relevance, a two-signal ranking pipeline re-orders results before presenting them to the user.

Normalization: Source-specific normalizers map heterogeneous API response schemas (Adzuna, Muse, JobSpy) into a common NormalizedJob dataclass (title, company, location, description, URL, salary, source). Jobs missing a title or description are dropped as they carry no ranking signal.

BM25 (lexical): Each job is represented as a concatenation of its title, company, location, and the first 500 characters of its description. BM25Okapi scores are computed against the user's query using word-level tokenization. This captures exact keyword overlap (job titles, skill names, location terms).

FAISS (semantic): On non-macOS deployments (or when HIREME_USE_FAISS=1), all job texts and the query are embedded in a single batched call using OpenAI text-embedding-3-small. L2-normalized vectors are indexed with faiss.IndexFlatIP for inner-product (cosine) similarity search. This captures semantic relevance where vocabulary diverges (e.g., "data analyst" vs. "business intelligence").

Reciprocal Rank Fusion (RRF): BM25 and FAISS rankings are fused using the formula:

$$\text{RRF_score}(d) = \sum_{r \in R} \frac{1}{k + \text{rank}_r(d)}$$

with $k = 60$. This rank-based combination is robust to score scale differences between the two signals. The top-N fused results are returned to the UI, with N configurable by the user (5–30).

3.4 Document Generation Pipeline

Once a candidate profile and job posting are available, the generation pipeline validates both inputs against their Pydantic schemas (CandidateProfile, JobPosting), then constructs two independent prompts using Markdown templates.

Resume prompt: Injects the full candidate JSON, the job posting JSON, and a Markdown resume template. The prompt includes explicit tailoring rules: prioritize content relevant to the job description, use metrics only when present in candidate data (no fabrication), and follow the template structure exactly.

Cover letter prompt: Injects the same candidate and job data alongside user-configurable preferences (tone, length, recipient name). Rules parallel the resume: no invented facts, evidence-only from candidate JSON, template-faithful output.

Both prompts include prompt injection defenses, instructing the model to treat all injected data as untrusted source material and to ignore commands embedded in job descriptions or candidate text.

Parallel generation: Resume and cover letter are generated concurrently using concurrent.futures.ThreadPoolExecutor(max_workers=2), reducing total wall time roughly by half compared to sequential calls. Both use gpt-4.1-nano with a user-adjustable temperature parameter (0.0–1.0), exposed in the UI as a "Precision ↔ Creative" slider.

Output formatting and export: Generated outputs are Markdown strings. Users can download either document as .md or as a .docx file. Word conversion uses pypandoc wrapping the pandoc binary; a pypandoc.download_pandoc() fallback auto-provisions the binary on deployments where it is not pre-installed (Streamlit Cloud installs it via packages.txt).

Conversion results are cached with @st.cache_data so repeated downloads do not re-invoke pandoc.

3.5 Architecture Summary

```
User Input
  ├── Upload Resume → format extractor → LLM parser → CandidateProfile
  └── Manual Form  → indexed widget state → CandidateProfile

Job Search (Page 2)
  └── LLM Agent → [Adzuna | Muse | JobSpy] → normalize → BM25 + FAISS → RRF → top-N

Document Generation
  └── CandidateProfile + JobPosting + Templates
      ├── ThreadPoolExecutor
      │   ├── Resume LLM → resume_output.md / .docx
      │   └── Cover Letter LLM → cover_letter_output.md / .docx
      └── Streamlit session state (cross-page persistence)
```

3.6 Overall Concepts Implemented

Concept	***Implementation***

Prompting	For both RAG, Resume/CV Generation we are prompting the LLMs to cater to our needs. Example for RAG Tool Call, below is our system prompt
	
	`(`
	`"You are a helpful job search assistant with three tools. "`
	`"- Adzuna: Best for keyword searches, salary filtering, and employment type (full-time, contract, etc.) "`
	`"- The Muse: Best for browsing by job category, experience level, or specific company. "`
	`"- JobSpy: Best for broad searches across LinkedIn, Indeed, Glassdoor, ZipRecruiter, and Google Jobs simultaneously. Use this when the user wants wide coverage, remote jobs, or when other tools return few results. "`
	`"You can use any combination of tools to broaden the`

	`scope and find more jobs, a single tool to narrow down results, or none."` `)`
RAG	We implemented RAG with normalizing the jobs and then fusing lexical and semantic scores to create a unified list of ranked results to return from our APIs.

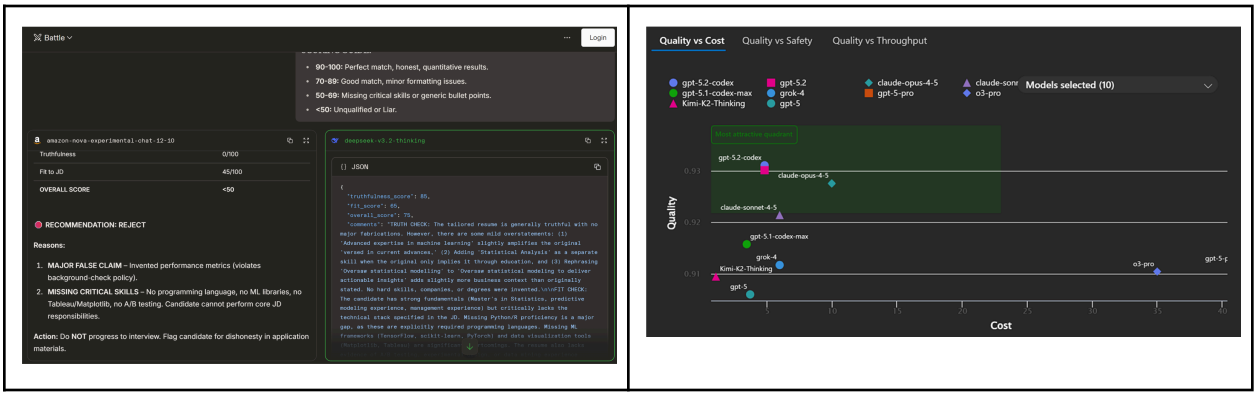
Tool Calls	Example of our JobSpy tool call:
	`@tool(`
	`"jobspy_search",`
	`parse_docstring=True,`
	`description=(`
	`"Search for job listings across multiple job boards (LinkedIn, Indeed, Glassdoor, ZipRecruiter, Google Jobs). "`
	`"Use this tool for broad searches across many sources at once, remote job filtering, or when other tools return few results."`
	`),`
	`)`

Multi-Modality	The user is able upload different forms for the resume data (text file, word doc, pdf, and even screenshot via jpeg)
APIs	We are using 3 main sources for jobs: Muse API, Adzuna API, Jobspy (web scraper “API”)
Scaling & Monitoring	We implemented Langchain to help add monitoring and compute cost of scaling. Our findings led to us parallelize some of the LLM calls, switching to lower latency models. Our rough estimate of cost is → 1k user interactions is ~$3.86, 1mil is ~$3,862

4 Experiments

During our testing, we evaluated a few different models and API. Overall, we initially started with testing between what model providers. We leveraged [Arena AI: The Official AI Ranking & LLM

Leaderboard, to test different models with example job resumes + jobs and asked them to evaluate. From Azure, we were also able to get a high level idea of cost vs quality between different models.



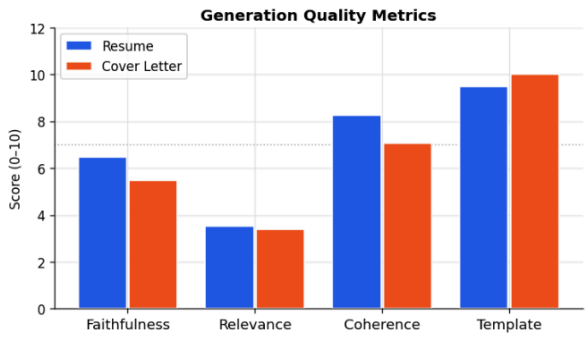
Our findings were that GPT 5 mini, nano were the best tradeoff between cost and quality. Example, switching to Grok-3 mini resulted in lower cost (and latencies) however it reduced quality by ~35% (score from 65/100 → 78/100).

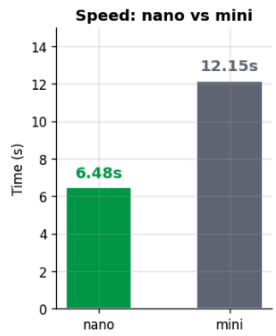
From there we started implementing our RAG and Resume/CV creation. A few different experiments:

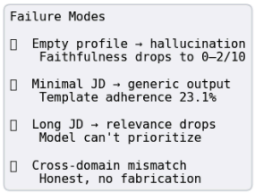
1. For RAG:
    a. We experimented first with using easily available APIs, Muse and Adzuna. However, we realized the quality of results was limited. The APIs did not provide a vast collection of jobs and often missed a lot of relevant jobs available on LinkedIn, GlassDoor, etc. Given these providers did not have an API, we leverage the Jobspy web scraper to increase our coverage of jobs.
    b. Then we experimented with the Tool calls and RAG setup. Initially the data from the jobs was very different and hard to rank. Hence we normalize it into a few general buckets such as title, description. Then we rank them and prioritize Jobspy results since they are higher coverage.
    c. We also experimented with the tool to ensure the model can select when to call all 3 tools. At the end, we briefly explain each tool to the system and then have it pick which one or combination of 3 to invoke.
2. For Resume/CV creation:
    a. We initially started with using GPT 5 nano to do the parsing, and creation of both Resume/CV in 1 LLM call. We quickly realized this led to very high latencies. Hence, we separated the calls to be individual LLM calls
    b. Then we parallelized the Resume and CV creation after the Resume parsing call was complete to help improve latency
    c. Additionally, we found that switching from GPT 5 nano to GPT 4.1 mini led to faster latencies and similar quality.

5 Results

We uploaded our app to streamlit cloud for users to try out and use: [Streamlit]. Also to help others understand and track our progress, work we created a GitHub Repo with documentation: [sidagarwal-labs/HireMe-AI: AI resume builder with job search]





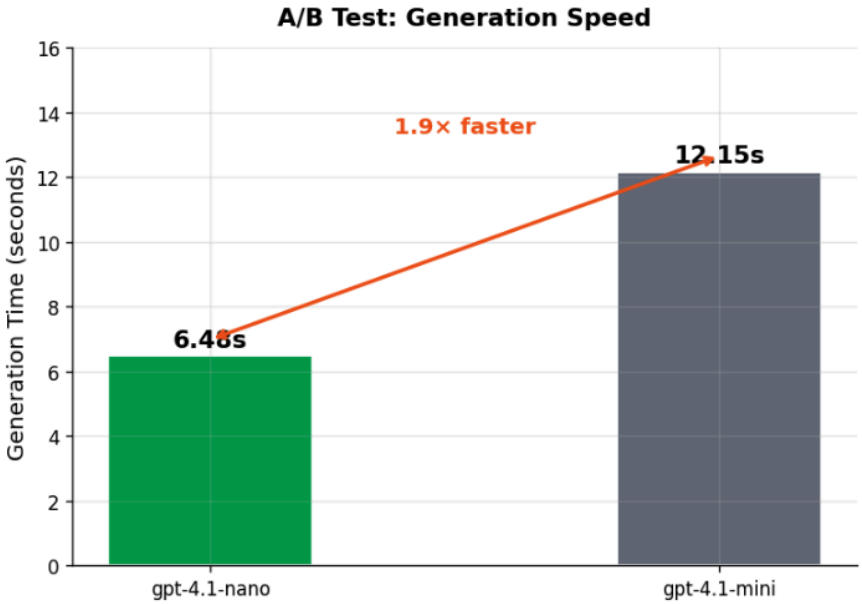


The LLM is instructed to not add skills and experience but this was only considered instructions not strong guard rails.

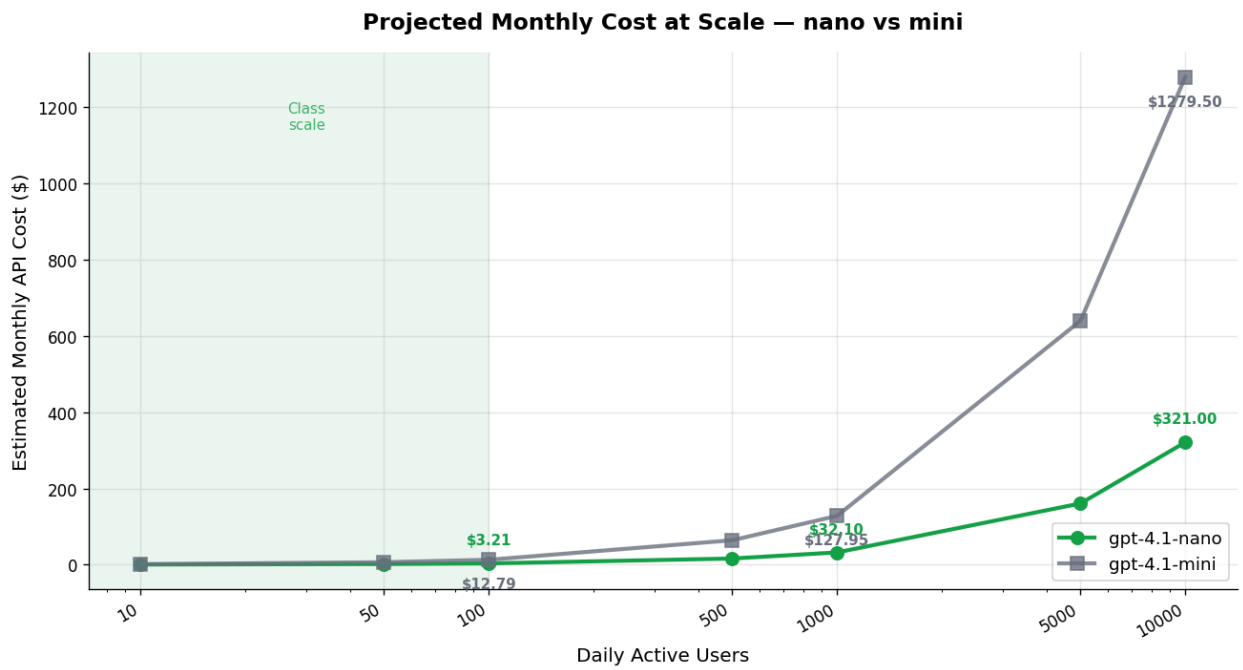
We then added such as:

“Require name, contact, and at least one of:
work experience, education, skills, projects.”

“Do not claim direct experience with a job requirement unless that skill, tool, or experience appears in the candidate profile.
You may express interest in learning or contributing, but do not imply prior experience.”



Nano is much cheaper for this use case since it is at a smaller scale. If we were to scale to more users as an actual product then we would switch models and request for paid api's from the job sites.



6 Conclusion and Future work

At the end, we were able to create a working pipeline of a user searching for some jobs, finding a job, then getting a tailored Resume and CV for it. However, some limitations of the system include:

1. We don't have a consistent and approved way to gathering jobs. The webscraper is not approved goes against TOS for LinkedIn, GlassDoor, etc. We would need to get the official API and cost.
2. We would like to setup a pipeline to do AB testing with real users to evaluate the quality of RAG. Today, it is hard to evaluate since we don't have a great idea of the expected results in jobs. Ideally, we would like to measure things like Click Rate, Abandonment Rate, etc on real live traffic to see if we are improving the ranking of results and jobs shown to the user. Also adding feedback mechanisms.

7 References

Adzuna API - Adzuna API

Muse API - Free Career Advice | Career Finder | Developers API

Jobspy API - rainmanjam/jobspy-api: Dockerized version of JobSpy job search utility with API key auth, rate limiting, and proxy support.

OpenAI API docs - OpenAI API Platform Documentation

RAG - [2402.03367] RAG-Fusion: a New Take on Retrieval-Augmented Generation

LLM Resume guide:
https://medium.com/@nayan.j.paul/building-resume-management-platform-using-genai-and-large-language-models-llms-8c9103a2ab15